

C02 - AT-1 Analytical Problem Solving

Name: V. Deepa Dharshini

Reg no: 192525213

Course Code: CSA0311

Course Name: Data Structure

21/1/20

- 1) Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately.

Setup:

Assume an array-based queue of fixed size 'n', and the operation pattern alternates 'enqueue, dequeue, enqueue... repeat 100 times.

Linear enqueue:

In a linear queue, **front** & **rear** only moving forward and are never wrapped back to the start of the array

- Every enqueue increments **rear**, every dequeue increments **front**
- The space is never reused because rear keeps moving

→ When $\text{rear} == n-1$, the queue reports "Queue Full" even though front has moved and there are empty space.

→ Result:- With 100 enqueue operations rear would need to advance 100 times. If array size n is smaller than 100, the linear queue fails (overflows) partway through.

Time complexity: $O(1)$, the issue is space utilization not speed.

Circular Queue:

In a circular queue, front & rear wrap around using

$$\text{rear} = (\text{rear} + 1) \% n$$

$$\text{front} = (\text{front} + 1) \% n$$

→ Space freed by dequeue is immediately reused because rear can wrap back to slot 0, 1, 2, ... and front has moved past them.

→ Result:- All 100 operations complete successfully with efficient, constant memory, whereas the linear queue with the small n would overflow.

Summary Table

Aspect	Linear Queue	Circular Queue
Memory reuse	No - freed front are wasted	Yes - slot reused via wraparound
Behaviour over 100 alternating	Fails/overflow once rear hits array end	Succeeds with small array
Time per operation	$O(1)$	$O(1)$
Space efficiency	Poor for long-running alternating workloads	Efficient - space bounded by max element
Practical Implication	Need $n \geq \text{total enqueue}$ to survive	Need $n \geq \text{max simultaneous element}$

Conclusion:

For alternating enqueue/dequeue workloads, a circular queue is far superior because it reclaims space dynamically. A linear queue's performance advantage disappears entirely - it simply runs out of room despite having free memory.

2) A queue implemented using an array reports "Queue Full" even though some positions are empty.

(a) Analyse why this occurs

(b) Explain how a circular queue solves this problem.

(a) Why this occurs :

→ Initially $\text{rear} = -1$ and $\text{front} = -1$ (or) 0

→ Each enqueue does $\text{rear} = \text{rear} + 1$ and inserts at that index

→ Each dequeue does $\text{front} = \text{front} + 1$.

→ Both pointers move only in one direction

→ So, once several dequeues have happened, the positions 0 to front are empty, but the queue's full check is typically:

if $(\text{rear} == n-1) \rightarrow \text{Queue Full}$

→ Since rear has already reached the last index of the array, the algorithm incorrectly reports the queue as full.

Root Cause :

The array positions before front are never reclaimed, rear has no way of circling back to reuse them, so usable space is permanently lost after each dequeue, even though it's physically empty.

ue) How a circular Queue Solves This
A circular queue treats the array as conceptually circular.

$$\text{rear} = (\text{rear} + 1) \% n$$

$$\text{front} = (\text{front} + 1) \% n$$

→ When rear reaches the last index $n-1$, and it needs to advance, $(n-1+1) \% n = 0$ so it wraps back to index 0 - provided that slot has already been vacated by a dequeue.

→ This means every position in array is reused as soon as it becomes free, instead of being permanently abandoned.

→ The condition: $\text{Full} = (\text{rear} + 1) \% n == \text{front}$

$$\text{Empty} = \text{front} == -1$$

Effect:

The queue only reports "Full" when all n slots are genuinely occupied with valid data - not merely because rear hit the end of the physical array. This fully utilizes the allocated memory and eliminates the false-full problem inherent in linear queues.